AUDIT TECHNIQUE

MoMoGuard

Plateforme de détection de fraude Mobile Money

Workspace : momoguard4/momoguard

Date : 21 juin 2026

1. Périmètre

Le dossier momoguard4/momoguard contient un seul projet : MoMoGuard. Le répertoire parent momoguard4 ne regroupe pas d'autres applications.

Élément	Détail
Nom	MoMoGuard — plateforme de détection de fraude Mobile Money
Contexte	Projet de fin d'études L4 — sécurité des utilisateurs
Cible	RDC — Orange Money, Airtel Money, Vodacom M-Pesa, Africell
Commits Git	1 commit principal

2. Architecture globale

Architecture client-serveur consentie :
• Agents (Windows Python, Android Kotlin prévu) → POST /report
• Serveur Flask + SocketIO traite, score et stocke
• Base SQLAlchemy (SQLite / PostgreSQL / MySQL)
• Dashboard web temps réel via WebSocket
• Firebase Auth + FCM pour l'app mobile

3. Langages et technologies

Couche	Techno	Fichiers clés
Backend	Python 3.12	app.py (519 l.), database.py (361), scoring.py (199)
Sécurité	Python	firebase_auth.py, threat_intel.py
Agent terrain	Python	agents/windows_agent.py (110 l.)
Frontend	HTML Jinja2	13 templates (~600 l.)
Styles	CSS	static/style.css (194 l.)
Temps réel	JavaScript	static/dashboard.js + Socket.IO
Base de données	SQLAlchemy 2.0	Schéma en Python
Conteneurisation	Docker Compose	Dockerfile, docker-compose.yml
Tests	pytest	8 fichiers, 28 tests
Prévu absent	Kotlin Android	Mentionné README, non présent

4. Composants du projet

4.1 Serveur central (app.py)

Framework : Flask 3 + Flask-SocketIO (threading).
Sécurité : auth admin, 2FA TOTP (pyotp), CSRF, rate limiting, en-têtes HTTP (CSP, X-Frame-Options, nosniff).
~30 routes : dashboard, API agents, utilisateurs Firebase, consentements, audit, paramètres.

4.2 Couche données (database.py)

ORM : SQLAlchemy Core. 7 tables : reports, consents, admins, agents, users, settings, audit_log.
Portabilité via MOMOGUARD_DB : SQLite (défaut), PostgreSQL (prod), MySQL (WAMP).
Migrations douces au démarrage sans perte de données.

4.3 Moteur de détection (scoring.py)

Méthode : scoring par règles explicables (pas de ML).
Signaux suspects → points → score 0-100 → niveau faible/moyen/élevé.
Seuils configurables : 25 (moyen) et 55 (élevé).
~20 règles : homoglyphes, liens suspects, urgence+lien, SIM swap, partage OTP, arnaques remboursement/loterie, fichiers exécutables, USSD.

4.4 Threat Intelligence (threat_intel.py)

API : Google Safe Browsing v4. Dégradation gracieuse sans clé API.

4.5 Firebase (firebase_auth.py)

Vérification ID tokens, notifications FCM sur alertes élevées. Optionnel sans serviceAccountKey.json.

4.6 Agent Windows (agents/windows_agent.py)

Surveillance consentie : presse-papier, dossier Téléchargements.
Libs : requests, pyperclip, watchdog. Envoi POST /report avec jeton.

4.7 Interface web

• dashboard.html — Alertes temps réel (WebSocket)

• agents.html — Gestion agents (bloquer, retirer)

• users.html — Utilisateurs Firebase (approbation, blocage)

• consent.html / consents.html — Consentement éthique

• audit.html — Journal d'audit

• settings.html — Seuils de scoring

• account.html — Compte admin + 2FA

5. Méthodes et approches

Domaine	Approche
Détection	Règles heuristiques explicables
Éthique	Consentement explicite, agent non furtif
Évaluation	Dataset labellisé, matrice de confusion, F1
Tests	pytest + fixtures Flask isolées
Déploiement	Docker Compose (PostgreSQL 16 + app)
Sécurité	2FA, CSRF, rate limit, audit log, headers
Temps réel	WebSocket vers dashboard
Multi-BDD	Abstraction SQLAlchemy par variable env

6. Résultats d'évaluation

Dataset : 48 messages (26 frauduleux, 22 légitimes).

Métrique	Valeur
Rappel (détection)	96,2 %
Faux positifs	0 %
Précision	100 %
F1-score	98 %
Exactitude	97,9 %

Faux négatif identifié : promo M-Pesa « envoyez 10$ et recevez 500$ ».

7. Dépendances Python

Web : Flask, Flask-SocketIO, Flask-WTF, Flask-Limiter
BDD : SQLAlchemy, PyMySQL, psycopg2-binary
Auth : pyotp, qrcode, firebase-admin
Agent : requests, pyperclip, watchdog
Tests : pytest

8. État actuel et points d'attention

Point	Statut
Agent Android Kotlin	Prévu README, absent du code
Tests pytest	28 tests — vérifier venv (requests)
serviceAccountKey.json	Ne pas committer
Migrations BDD	Manuelles, pas d'Alembic
ML / IA	Aucun — 100 % rule-based

Synthèse

MoMoGuard est une plateforme Python/Flask de cybersécurité orientée Mobile Money (RDC), avec un moteur de détection par règles explicables, un agent Windows, un dashboard temps réel, une couche Firebase pour le mobile, du Docker pour la production, et une méthodologie d'évaluation académique. Le client Android Kotlin reste à implémenter ou à ajouter au dépôt.